MLSP Cheatsheet: HW 6

Nitish Vikas Deshpande and Japneet Singh

November 2019

1 Maths refresher and DSP

Probability

- **Probability Rules**
  :
  - Marginalization or Sum Rule: $p(x) = \int p(x,y)dy$ for Continuous Random Variable and $P(w) = \sum_t P(t,w)$ for Discrete Random Variable
  - Product Rule: $P(w,t) = P(w|t)P(t)$
  - Bayes Rule: $P(t|w) = \frac{P(t,w)}{P(w)} = \frac{P(w|t)P(t)}{P(w)}$
- **Probability Distribution or CDF**
  : $P(z) = \int_{-inf}^{z} p(x)dx$
- **Transformation of Variables**
  : $p_y(y) = \sum_i \frac{p_x(x)}{\left|\frac{dy}{dx}\right|}\Bigg|_{x_i}$
- **Expectations**
  :
  - Mean: $E[f(x)] = \int f(x)p(x)dx$
  - Variance: $\mathrm{var}[f(x)] = E\left[(f(x) - E[f(x)])^2\right]$
  - Conditional Expectation: $E_x[f(x)|y] = \int f(x)p(x|y)dx$
  - Covariance: $\mathrm{cov}[\boldsymbol{x},\boldsymbol{y}] = E\left[(\boldsymbol{x} - E[\boldsymbol{x}])\left(\boldsymbol{y}^T - E\left[\boldsymbol{y}^T\right]\right)\right]$
  - Correlation: $\rho_{x,y} = \mathrm{corr}(x,y) = \frac{\mathrm{cov}[x,y]}{\sigma_x \sigma_y}$

Matrix Calculus

- $\frac{\partial}{\partial w}(\Phi u) = \frac{\partial u}{\partial w}\Phi^T$
- $\frac{\partial}{\partial \boldsymbol{w}}\left(\boldsymbol{u}^T\boldsymbol{v}\right) = \frac{\partial \boldsymbol{u}}{\partial \boldsymbol{w}}\boldsymbol{v} + \frac{\partial \boldsymbol{v}}{\partial \boldsymbol{w}}\boldsymbol{u}$

Similarity Measures

Euclidean distance

$$d(x_1,x_2) = |x_1 - x_2| = \sqrt{\sum_n (x_1[n] - x_2[n])^2}$$

Cosine Similarity

$$d(\boldsymbol{x}_1,\boldsymbol{x}_2) = \frac{\boldsymbol{x}_1^T \cdot \boldsymbol{x}_2}{|\boldsymbol{x}_1|\,|\boldsymbol{x}_2|} = \cos(\theta)$$

KL Divergence

$$d(p_1(x), p_2(x)) = -\sum_x p_1(x)\log\left(\frac{p_2(x)}{p_1(x)}\right)$$

Digital Signal Processing

- **Convolution**
  : $y[n] = x[n] * h[n] = \sum_m x[m]h[n-m]$
- **Cross Correlation**
  : $y[n] = x[n] \otimes h[n] = \sum_m x[m]h[m+n]$
- **Fourier Transform Synthesis equation**
  :
  $x(t) = \sum_k X[k]e^{jk\omega_0 t}X$
- **Fourier Transform Analysis equation**
  :
  $X_k = \frac{1}{T}\int x(t)e^{-jk\omega_0 t}dt$
- **Short Time Fourier Transform**
  :
  $X[k,m] = \sum_{n=0}^{N-1} x[n]w[n-mH]e^{-j\frac{2\pi}{N}kn}$;
  $k = 0, 1, \ldots, N-1$

2 Classification

Classification

Comparison parameters	Multi-class classification	Multi-label classification
Random Variable	$c \in \{1,\ldots,N\}$	$l_i \in \{0,1\}$, for $i \in \{1,\ldots,N\}$
Ground Truth PDF	$t[c]$	$t_i[l_i]$
Estimated PDF	$y[c]$	$y_i[l_i]$
Metric to Minimize	$H(t[c], y[c])$	$H(t_i[l_i], y_i[l_i])$

Neural Network: Backpropogation

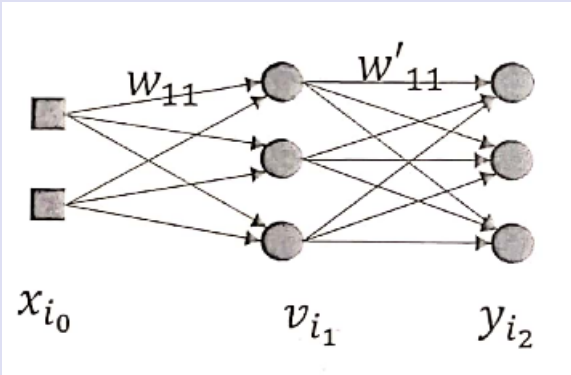

Figure 1: Neural Network with 1 hidden layer

$h_{i_1} = \sum_{i_0=1}^{2} w_{i_0 i_1} x_{i_1}$
$v_{i_1} = \mathrm{sigmoid}\left(h_{i_1}\right)$
$h'_{i_2} = \sum_{i_1=1}^{3} w'_{i_1 i_2} x_{i_1}$
$y_{i_2} = \mathrm{softmax}\left(h'_{i_2}\right)$

Using Categorical Cross entropy loss $E = -\sum_{i2} t_{i2}\log y_{i2}$ the updates for the weights in backpropogation can be derived using the following gradients

- $\frac{\partial E}{\partial w'_{11}} = (y_1 - t_1)\,v_1$
- $\frac{\partial E}{\partial \omega_{11}} = x_1 v_1 (1 - v_1)\left(\sum_{i_2} \omega'_{1i_2}\left(y_{i_2} - t_{i_2}\right)\right)$

Neural Networks: Optimisation

Method	Description
Adding Noise improves generalisation	$\{0,1\} \to \left\{\frac{\epsilon}{1-K}, 1-\epsilon\right\}$
Early stopping	Limits the search region volume to $\propto \eta\tau$ where η is the learning rate and τ is the step size
Ensemble models or Bagging	Final output is the average of many models
Dropout	Mask neurons randomly

-

Evaluation Measures for Classification

1. **Accuracy** Accuracy= $\frac{\text{correct}}{all} = \frac{TP+TN}{TP+TN+FP+FN}$
2. **Precision**
$$P = \frac{TP}{TP+FP}$$
3. **Recall**
$$R = \frac{TP}{TP+FN}$$
4. **F measure**
$$F = \text{harmonic mean } (P, R) = \left(\frac{P^{-1}+R^{-1}}{2}\right)^{-1}$$

Fisher's linear discriminant method

Maximize the separation between means, but Minimize the intra-class variance
$w = S_w^{-1}(m_2 - m_1)$ where
$\mathbf{S}_\mathrm{W} = \sum_{n\in\mathcal{C}_1}(\mathbf{x}_n - \mathbf{m}_1)(\mathbf{x}_n - \mathbf{m}_1)^\mathrm{T} + \sum_{n\in\mathcal{C}_2}(\mathbf{x}_n - \mathbf{m}_2)(\mathbf{x}_n - \mathbf{m}_2)^\mathrm{T}$
and
$\mathbf{m}_1 = \frac{1}{N_1}\sum_{n\in\mathcal{C}_1}\mathbf{x}_n \quad \mathbf{m}_2 = \frac{1}{N_2}\sum_{n\in\mathcal{C}_2}\mathbf{x}_n$

Data Annotation

Cohen's Kappa score:
$$\kappa = \frac{P(y=y') - P(y)P(y')}{1 - P(y)P(y')}$$

3 Regression

General Loss Function:
$E = (t - \Phi\omega)^\top(t - \Phi\omega) + \lambda\sum_j|\omega_j|^q$
Lasso for q=1
Ridge regression for q=2

Least Square Solution

: $\omega = \left(\Phi^\top\Phi\right)^{-1}\Phi^\top t$

L2 or Ridge Regression

: $\omega = \left(\lambda I + \Phi^\top\Phi\right)^{-1}\Phi^\top t$

Gradient Descent

$w^{(\tau+1)} = w^{(\tau)} - \eta\frac{\partial E}{\partial w}$
where τ is iteration index and η is the learning rate.
Use iterative updates when analytical solution is not possible or when matrix inversion based solutions become difficult because data is arriving in a stream.
Momentum in Gradient Descent: $v \leftarrow \beta v + (1-\beta)\Delta w$
$W = W - \alpha v$

1 Batch	Mini Batch	N batches
N samples	Stochastic Gradient Descent: Randomly sampled mini batches	Online method(1 sample per batch)

Trick: Start with small batch size and increase it with epochs

4 Parameter Estimation

Maximum Likelihood Estimate (MLE)

$$\mu, \sigma = \underset{\mu,\sigma}{\operatorname{argmax}} \ln p(t_1, \ldots, t_N|\mu, \sigma)\Big|_{t_1=s_1,\ldots,t_N=s_N}$$

Maximum a-posteriori estimate (MAP)

$$\mu, \sigma = \underset{\mu,\sigma}{\operatorname{argmax}} \ln p(\mu, \sigma|t_1, \ldots, t_N)\Big|_{t_1=s_1,\ldots,t_N=s_N}$$

Expectation Maximization Algorithm

- E step: $q \leftarrow \operatorname{argmin}_q \mathrm{KL}[q(Z)\|p(Z|X,\theta)]$
- M step: $\theta \leftarrow \operatorname{argmax}_\theta E_q[\log p(X, Z|\theta)]$

5 Latent Variable Models

K Means Clustering

- $\mathcal{L} = \sum_{n,k} r_{nk}\|x_n - \mu_k\|^2$
- Given r_{nk}, $\frac{\partial L}{\partial \mu_{k'}} = 2\sum_n r_{nk'}(x_{k'} - \mu_{k'}) = 0$
$\Rightarrow \quad \mu_{k'} = \frac{\sum_n r_{nk'}\mathcal{L}_{k'}}{\sum_n r_{nk'}}$
- Given μ_k, $r_{nk} = \begin{cases} 1, & \text{if } k = \operatorname{argmin}_k\|x_n - \mu_k\| \\ 0, & Otherwise \end{cases}$

Gaussian Mixture Model

- $\mu_{k'} = \frac{\sum_n \gamma_{nk'} s_n}{N_{k'}}$
- $\Sigma_k = \frac{1}{N_k}\sum_n \gamma_{n_k}(s_n - \mu_k)(s_n - \mu_k)^\top$
- $\pi_{k'} = \frac{N_k}{N}$

6 Dimensionality Reduction

Principle Component Analysis

1. Given data samples $\boldsymbol{s}_n \in R^D$
2. Normalize: $\boldsymbol{x}_n = \boldsymbol{s}_n - E[\boldsymbol{s}]; \quad E[\boldsymbol{s}] = \frac{1}{N}\sum_{n=1}^N \boldsymbol{s}_n$
3. Obtain variance matrix $S = \frac{1}{N}\sum_{n=1}^N \boldsymbol{x}_n\boldsymbol{x}_n^T$
4. Eigen value decomposition of S to get $\lambda_i, \boldsymbol{u}_i; i = 1, \ldots, D$ with λ_i in decreasing order
5. Choose first M eigen vectors as the principal axes
6. $\widetilde{\boldsymbol{x}}_n = \sum_{i=1}^M \left(\boldsymbol{x}_n^T\boldsymbol{u}_i\right)\boldsymbol{u}_i$

Non Negative Matrix Factorisation

1. $W_{ir} \leftarrow W_{ir}\frac{\left(VH^T\right)_{ir}}{\left(WHH^T\right)_{ir}}$
2. $H_{rj} \leftarrow H_{rj}\frac{\left(W^TV\right)_{rj}}{\left(W^TWH\right)_{rj}}$

Probabilistic Latent Component Analysis

1. E step: $q_t(z|f) = \frac{P_t(z)P(f|z)}{\sum_z P_t(z)P(f|z)}$
2. M step: $P_t(z) = \frac{\sum_f V_{ft}q_t(z|f)}{\sum_{z'}\sum_f V_{ft}q_t(z'|f)}$
$P(f|z) = \frac{\sum_t V_{ft}q_t(z|f)}{\sum_{ft} V_{ft}q_t(z|f)}$

Nitish Vikas Deshpande(170450)
Japneet Singh (170313)